SIMATS ENGINEERING
ASSESSMENT TOOL 1: Constraint-Based Problem Solving

COURSE NAME: COMPUTER VISION COURSE CODE: ITA0515 Slot B

COURSE OUTCOME COVERED
CO5: ***Develop computer vision applications using OpenCV for performing recognition and analysis on images and videos.*** **(BTL 6)**

Assessment Tool Weightage: 40%

QUESTIONS:5 Total Marks 50 Duration :60 Minutes Annexure A
Constraint-Based Problem Solving

Code of Conduct:

I R Dinesh-192421117 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. IL= understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: R Dinesh

Rubrics for Evaluation

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	*Marks(100)*
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly	
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization	
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints	
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts	

Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided	

Constraint-Based Problem Solving: Analyze the problem and design an end-to-end computer vision pipeline using OpenCV, clearly identifying each stage from input acquisition to final output. Ensure that your solution satisfies all given constraints and justify your design decisions at each stage.

1. Real-Time Face Recognition System (Edge Deployment)
A company wants to deploy a face recognition system on low-power edge devices (e.g., Raspberry Pi). The system must:
A. Process live video streams in real time (<30 FPS latency)
B. Work under varying lighting conditions
C. Use limited memory and CPU resources
D. Detect and recognize faces from a predefined dataset
Design a complete solution using OpenCV, specifying: image acquisition, preprocessing, feature detection, and recognition pipeline. Justify how your design satisfies all constraints.

2. Automated Video Surveillance with Alert System
A security firm needs a video surveillance system that:
A. Detects motion and unusual activities
B. Generates alerts in real time
C. Works with standard CCTV video feeds
D. Minimizes false positives in dynamic environments
Design an OpenCV-based system integrating video processing, feature detection, and motion analysis. Justify how your approach handles noise, dynamic backgrounds, and real-time constraints.

3. OCR System for Low-Quality Documents
A digitization company needs an OCR system that:
A. Extracts text from low-resolution, noisy scanned images
B. Handles varying fonts and illumination
C. Works efficiently for batch processing
Design an image processing pipeline using OpenCV functions (enhancement, segmentation, etc.) for OCR. Justify how each stage improves recognition under given constraints.

4. Facial Expression Recognition for Mobile Application
A mobile app must detect facial expressions (happy, sad, neutral) with the following constraints:
A. Limited processing power (mobile CPU)
B. Real-time performance
C. Robustness to slight head movement and lighting changes
Design a solution using OpenCV for face detection and expression analysis. Justify how your approach balances accuracy and efficiency.

5. Smart Traffic Monitoring System
A city authority needs a system to:
A. Detect vehicles from live video streams

B. Track movement across frames
C. Count vehicles and display statistics
D. Operate continuously with minimal delay
Design a complete OpenCV-based system including video handling, detection, tracking, and visualization (drawing functions). Justify how your system ensures accuracy and real-time performance.

1. Real-Time Face Recognition System – Edge Deployment

Requirements

- Real-time processing
- Varying lighting
- Low CPU and memory
- Recognize faces from a predefined dataset

Proposed Pipeline

Camera → Preprocessing → Face Detection → Feature Extraction → Recognition → Output

1. Image Acquisition

Use OpenCV:
cv2.VideoCapture(0)
to capture live video from the camera.

2. Preprocessing

- Convert frame to grayscale.
- Resize the frame to a smaller resolution.
- Apply histogram equalization or CLAHE to improve lighting variations.
- Apply slight Gaussian filtering to reduce noise.

3. Face Detection

Use **Haar Cascade** or a lightweight OpenCV face detector.
face_cascade.detectMultiScale()
This is suitable for low-power devices because it requires less computational power.

4. Feature Extraction

Extract facial features from the detected face using a lightweight method such as **LBPH (Local Binary Pattern Histogram)**.

5. Recognition

Compare the extracted features with the predefined face database using an **LBPH Face Recognizer**.

6. Output

Draw a rectangle around the detected face and display the person's name.

Why this satisfies the constraints

- **Real-time:** Resize frames and use lightweight Haar + LBPH methods.
- **Lighting:** Grayscale and CLAHE improve robustness.
- **Low resources:** Avoid heavy deep-learning models.
- **Recognition:** LBPH compares detected faces with the predefined dataset.

Final Pipeline

Camera → Resize → Grayscale → CLAHE → Haar Face Detection → LBPH Feature Extraction → Face Matching → Name Display

---

2. Automated Video Surveillance with Alert System

Requirements

- Detect motion/unusual activity
- Real-time alerts

- Standard CCTV feeds
- Minimize false positives

Proposed Pipeline

CCTV Video → Frame Acquisition → Preprocessing → Background Subtraction → Morphology → Motion Detection → Analysis → Alert

1. Video Acquisition

Use:
cv2.VideoCapture()
to receive CCTV video.

2. Preprocessing

- Resize frames to reduce processing time.
- Convert to grayscale.
- Apply Gaussian blur to remove noise.

3. Motion Detection

Use **Background Subtraction** such as:
cv2.createBackgroundSubtractorMOG2()
It identifies moving objects by comparing the current frame with the background.

4. Morphological Processing

Apply:

- Opening → removes small noise.
- Closing → fills small gaps.

This improves the quality of detected motion regions.

5. Feature/Activity Analysis

Find contours using:
cv2.findContours()
Then calculate the area and bounding box of moving objects.
Very small regions are ignored to reduce false alarms.

6. Alert Generation

If a moving object remains above a specified area threshold for several consecutive frames, generate an alert.
Example:
Motion detected → Verify for several frames → Alert

Why this satisfies the constraints

- **Real-time:** Background subtraction is computationally efficient.
- **Noise:** Gaussian filtering and morphology remove noise.
- **Dynamic backgrounds:** MOG2 adapts to background changes.
- **False positives:** Area and consecutive-frame checks prevent alerts from small temporary movements.

Final Pipeline

CCTV → Resize → Grayscale → Gaussian Blur → MOG2 → Morphological Filtering → Contour Detection → Motion Verification → Alert

---

3. OCR System for Low-Quality Documents

Requirements

- Low-resolution and noisy images
- Different fonts
- Uneven illumination

- Efficient batch processing

Proposed Pipeline

Document Image → Enhancement → Noise Removal → Thresholding → Morphology → Text Segmentation → OCR → Text Output

1. Image Acquisition

Read scanned documents using:
cv2.imread()

2. Image Enhancement

Convert to grayscale:
cv2.cvtColor()
Improve contrast using **CLAHE**.

3. Noise Removal

Use:
cv2.GaussianBlur()
or median filtering to reduce scanning noise.

4. Segmentation

Apply **adaptive thresholding**:
cv2.adaptiveThreshold()
This works better than a fixed threshold when illumination varies across the document.

5. Morphological Processing

Use morphological **opening** to remove small noise and **closing** to connect broken characters.

6. Text Region Detection

Use contours to identify text regions and arrange them in reading order.

7. OCR

Send the processed image to an OCR engine such as **Tesseract**.

8. Output

The recognized text is stored in a text file or database.

Why this satisfies the constraints

- **Low quality:** Upscaling, contrast enhancement, and denoising improve readability.
- **Varying illumination:** Adaptive thresholding handles local brightness differences.
- **Different fonts:** OCR engine can recognize multiple font styles.
- **Batch processing:** Images can be processed sequentially using the same pipeline.

Final Pipeline

Scanned Image → Grayscale → CLAHE → Denoising → Adaptive Threshold → Morphology → Text Segmentation → OCR → Text Output

---

4. Facial Expression Recognition for Mobile Application

Requirements

- Limited CPU
- Real-time processing
- Slight head movement
- Lighting variations
- Recognize Happy, Sad and Neutral

Proposed Pipeline

Camera → Resize → Face Detection → Face Alignment → Preprocessing → Feature Extraction → Expression Classification → Output

1. Image Acquisition

Capture live video using:
cv2.VideoCapture(0)

2. Preprocessing

- Resize the frame.
- Convert to grayscale.
- Apply CLAHE for lighting normalization.
- Use a small Gaussian blur to reduce noise.

3. Face Detection

Use a lightweight Haar Cascade detector:
detectMultiScale()

4. Face Alignment

Use the detected face region and normalize its size.
This reduces the effect of slight head movement.

5. Feature Extraction

Extract facial features such as:

- Eyes
- Eyebrows
- Mouth
- Facial contours

A lightweight feature-based classifier can then be used.

6. Expression Classification

Classify the face into:
Happy
Sad
Neutral
A lightweight machine-learning classifier such as **SVM** can be used for mobile CPU constraints.

7. Output

Display:
Expression: Happy
on the video frame.

Why this satisfies the constraints

- **Low CPU:** Haar detection and lightweight classification are efficient.
- **Real-time:** Resize frames and process only the face region.
- **Head movement:** Face normalization/alignment improves stability.
- **Lighting:** CLAHE improves contrast under changing illumination.

Final Pipeline

Camera → Resize → Grayscale → CLAHE → Haar Face Detection → Face Alignment → Feature Extraction → SVM Classification → Expression Display

---

5. Smart Traffic Monitoring System

Requirements

- Detect vehicles from live video
- Track vehicles across frames
- Count vehicles
- Display statistics
- Continuous real-time operation

Proposed Pipeline

Camera/CCTV → Frame Acquisition → Preprocessing → Vehicle Detection → Tracking → Counting → Visualization → Statistics

1. Video Acquisition

Use:
cv2.VideoCapture()
to capture live traffic video.

2. Preprocessing

- Resize frames.
- Apply Gaussian filtering if required.
- Improve image quality under poor lighting.

3. Vehicle Detection

Detect:

- Cars
- Buses
- Trucks
- Motorcycles

A lightweight object detector such as YOLO can be used.

4. Vehicle Tracking

Assign an ID to each detected vehicle and track it across consecutive frames.
Example:
Car ID 1
Car ID 2
Truck ID 3
Tracking prevents the same vehicle from being counted repeatedly.

5. Vehicle Counting

Draw a virtual counting line:
---------- COUNTING LINE ----------
When a vehicle crosses the line, increase the counter.

6. Visualization

Use OpenCV drawing functions:
cv2.rectangle()
cv2.line()
cv2.putText()
to display:

- Bounding boxes
- Vehicle IDs
- Counting line
- Vehicle count
- FPS

7. Statistics

Display:
Cars: 25
Buses: 5
Trucks: 8
Total Vehicles: 38
FPS: 30

Why this satisfies the constraints

- **Detection:** Object detection identifies vehicles.

- **Tracking:** IDs maintain vehicle identity across frames.
- **Accurate counting:** Line-crossing prevents duplicate counting.
- **Real-time:** Resize frames and use an efficient detector/tracker.
- **Visualization:** OpenCV drawing functions provide live statistics.

Final Pipeline

CCTV/Camera → Resize → Preprocessing → Vehicle Detection → Object Tracking → Line Crossing → Vehicle Counting → Statistics & Visualization